부록 · LangChain4j · Spring AI

LangChain4j + Spring Boot Java · Kotlin 비교 학습자료

업비트 XRP 시세 분석 · Ollama Gemma3 연동 예제

Ollama 서버:

```text
http://localhost:11434
```

1. 전체 구조

업비트 API → XRP(KRW-XRP) 시세/캔들

→ Spring Boot → LangChain4j

→ Ollama (http://localhost:11434) → Gemma3

→ 상승 / 하락 / 횡보 분석 → REST API 결과

처음에는 DB, JPA, Repository를 넣지 않고 '업비트 데이터 → AI 분석 → REST API 출력' 흐름부터 성공시키는 것이 가장 이해하기 쉽습니다.

2. Spring Boot Java + Kotlin 비교표

항목	Spring Boot Java	Spring Boot Kotlin
코드 길이	비교적 김	짧고 간결함
LangChain4j	매우 좋음	매우 좋음
Ollama / Gemma3	연동 가능	연동 가능
업비트 REST API	연동 가능	연동 가능
DTO	class / record	data class
Controller	조금 길어짐	간결함
JPA	자료가 매우 많음	사용 편리
학습 자료	매우 많음	많음
추천 용도	기본 원리 학습	주력 개발에 추천

3. 핵심 구성요소 3개

① UpbitService — 업비트에서 XRP 시세와 캔들 데이터를 가져옴

② XrpAiService — LangChain4j를 통해 Gemma3에게 분석을 요청함

③ XrpController — 브라우저나 앱에서 호출할 REST API를 제공함

4. 공통 application.properties

```properties
server.port=8080

ollama.base-url=http://localhost:11434
ollama.model=gemma3
```

base-url은 Ollama가 실행 중인 서버 주소이고, model은 Ollama에 설치된 정확한 모델명을 지정합니다. 실제 설치 이름이 gemma3:4b 등이라면 그 이름으로 변경합니다.

5. Java 예제

5-1. OllamaConfig.java

```java
@Configuration
public class OllamaConfig {

    @Bean
    public ChatModel chatModel(
            @Value("${ollama.base-url}") String baseUrl,
            @Value("${ollama.model}") String modelName) {

        return OllamaChatModel.builder()
                .baseUrl(baseUrl)
                .modelName(modelName)
                .temperature(0.2)
                .build();
    }
}
```

5-2. XrpAiService.java

```java
@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String predict(String priceData) {

        String prompt = """
                당신은 암호화폐 시세 분석 AI입니다.

                다음은 업비트 XRP 가격 데이터입니다.

                %s

                현재 추세:
                예상 방향: 상승/하락/횡보
                이유:
                주의사항:
                """.formatted(priceData);

        return chatModel.chat(prompt);
    }
}
```

5-3. XrpController.java

```java
@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final XrpAiService aiService;

    public XrpController(XrpAiService aiService) {
        this.aiService = aiService;
    }

    @GetMapping("/predict")
    public String predict() {

        String data = """
                현재가: 4,200원
                5일 이동평균: 4,150원
                20일 이동평균: 4,000원
                거래량: 전일 대비 35% 증가
                RSI: 62
                """;

        return aiService.predict(data);
    }
}
```

6. Kotlin 예제

6-1. OllamaConfig.kt

```kotlin
@Configuration
class OllamaConfig {

    @Bean
    fun chatModel(
        @Value("\${ollama.base-url}") baseUrl: String,
        @Value("\${ollama.model}") modelName: String
    ): ChatModel {

        return OllamaChatModel.builder()
            .baseUrl(baseUrl)
            .modelName(modelName)
            .temperature(0.2)
            .build()
    }
}
```

6-2. XrpAiService.kt

```kotlin
@Service
class XrpAiService(
    private val chatModel: ChatModel
) {

    fun predict(priceData: String): String {

        val prompt = """
            당신은 암호화폐 시세 분석 AI입니다.

            다음은 업비트 XRP 가격 데이터입니다.

            $priceData

            현재 추세:
            예상 방향: 상승/하락/횡보
            이유:
            주의사항:
        """.trimIndent()

        return chatModel.chat(prompt)
    }
}
```

6-3. XrpController.kt

```kotlin
@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val aiService: XrpAiService
) {

    @GetMapping("/predict")
    fun predict(): String {

        val data = """
            현재가: 4,200원
            5일 이동평균: 4,150원
            20일 이동평균: 4,000원
            거래량: 전일 대비 35% 증가
            RSI: 62
        """.trimIndent()

        return aiService.predict(data)
    }
}
```

7. Java ↔ Kotlin 핵심 차이

Java	Kotlin
private final ChatModel chatModel;	private val chatModel: ChatModel
public String predict(String data)	fun predict(data: String): String
생성자를 별도로 작성	주 생성자에서 바로 의존성 주입
코드가 명시적이고 길어짐	같은 기능을 더 짧게 작성

8. 실제 업비트 XRP 데이터로 발전시키기

위 예제는 학습을 위해 가격 데이터를 직접 넣었습니다. 실제 프로젝트에서는 UpbitService가 KRW-XRP 캔들을 가져오고 IndicatorService가 MA5, MA20, RSI, 거래량 등을 계산한 뒤 그 결과를 Gemma3에 전달합니다.

Upbit REST API → KRW-XRP 최근 캔들 수집

IndicatorService → MA5 / MA20 / RSI / 거래량 계산

XrpAiService → 계산된 지표를 Gemma3에 전달

Gemma3 → 상승 / 하락 / 횡보와 이유 분석

XrpController → JSON 형태로 결과 반환

9. 권장 프로젝트 구조

```text
upbit-xrp-ai/
├─ controller/
│  └─ XrpController.kt
├─ service/
│  ├─ UpbitService.kt
│  ├─ IndicatorService.kt
│  └─ XrpAiService.kt
├─ config/
│  └─ OllamaConfig.kt
├─ dto/
│  ├─ CandleDto.kt
│  └─ PredictionDto.kt
└─ UpbitAiApplication.kt
```

10. 실행 흐름

```text
GET http://localhost:8080/api/xrp/predict

        ↓
UpbitService
        ↓
IndicatorService
MA5 / MA20 / RSI / 거래량
        ↓
LangChain4j
        ↓
Ollama
http://localhost:11434
        ↓
Gemma3
        ↓
상승 / 하락 / 횡보 분석
        ↓
REST API 결과
```

11. 학습 순서

1단계: Ollama + Gemma3 연결 → 2단계: LangChain4j chat() 호출 → 3단계: REST Controller 작성 → 4단계: 업비트 실제 데이터 연결 → 5단계: 기술지표 계산 → 6단계: DTO로 JSON 결과 정리 → 7단계: 필요할 때 MariaDB/JPA를 추가합니다.

주의: LLM인 Gemma3의 답변을 미래 가격의 확정적 예측으로 사용하면 안 됩니다. 기술지표와 시장 데이터를 해석·요약하는 보조 분석 도구로 사용하는 것이 적절합니다.
